

$\pi_{0.5}$ · arXiv 2504.16054 · Physical Intelligence

Visionary

11+3 · seat 1 = Kanta

2026 BEHAVIOR map · Oct 2 deltas
Keep FM · hierarchy · RTC · VI

Sep 30 paper club · VLA Architectures

Spine: open-world + hybrid FAST discrete pretrain → FM retained

Semantic subtask → FM chunk (~50-step/~1s @ 50 Hz) · NOT a classical planner · RTC companion

Club spine · one-liner

Open-world VLA via heterogeneous co-training + hierarchical inference:

FAST discrete pretrain → FM retained for continuous control.

Semantic subtask → ~50-step / ~1s FM chunk @ 50 Hz. NOT a classical planner.

RTC companion for async soft-inpaint. Blogs: “FAST replaced FM” = FALSE.

BEHAVIOR'26: $\pi 0.5$ + GR00T baselines · Oct 2 = $\pi 0.5$ -baseline + deltas.

FM vs FAST · say this aloud

Hybrid FAST discrete pretrain → FM retained for continuous control

Claim	Verdict
FAST replaced FM	MYTH
FAST pretrain + FM post-train	TRUE
FM head retained at inference	TRUE
OpenPI FM API for $\pi_{0.5}$	CLEAR
FAST is the deployed LL controller	FALSE

OpenPI ships FM head for $\pi_{0.5}$ · KI (2505.23705) formalizes stop-grad FAST+FM

Hybrid recipe · FAST discrete pretrain → FM retained

Bust the myth: FAST does NOT replace FM at deploy

① PRE-TRAIN · 280k steps

Discrete next-token VLA

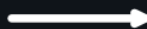
SigLIP 400M + Gemma ~2B

text · boxes · FAST action tokens · $\alpha=0$

FAST action tokens

scalable discrete pretrain signal

Mixture: MM · ME · CE · HL · WD



② POST-TRAIN / INFER · 80k

Joint NTP + flow matching $\alpha=10$

Action expert ~300M · FM

Ada-RMSNorm on τ · random init expert

Inference: AR subtask → 10 FM steps

~50-step / ~1s chunk @ 50 Hz PD

MM+ME+WD+VI · drop CE · FM RETAINED

NOT classical planner · same model for HL + LL · RTC = async soft-inpaint companion (2506.07339)

Hierarchical control loop · semantic subtask → FM chunk

Outer semantic loop · inner continuous control · NOT a classical planner



Control knobs (paper vs challenge priors)

Train H	Hz	NFE	HL runtime	Cams
Paper 50 Comet ~32	Paper 50 Comet ~30	Paper 10 sweep ≤ 10	On (same wts) vs off / oracle	HL: all 4 LL: wrists+fwd

2026 BEHAVIOR map · CLEAR

Official baselines · scale · timeline · '25 preview

Official baselines

π 0.5 + GROOT N1.7

behavior.stanford.edu/challenge

Scale

100 tasks · 7 scenes · 20k demos

~1950 h · RGB+depth+proprio · BDDL

Timeline 2026

07/02



launch

10/16



submit

11/04



winners

'25 winner on Pi0.5 · preview of '26 bottlenecks

RLC adapted Pi0.5 → ~26% q (correlated FM noise · System-2 stages · multi-sample FM · compression)

Bottlenecks: precision grasps · non-Markov stages · recovery demos · Oct 2 → propose deltas on this substrate

Oct 2 proposal framing · π 0.5-baseline + targeted deltas

NOT greenfield · mirrors '25 winning pattern + challenge starter kit

1

Keep FM

FAST/KI for reps & language
never “replace FM with FAST”

HIGH
LEVERAGE

2

Hierarchy / stages

Paper HL + '25 System-2
non-Markov BEHAVIOR states

HIGH
LEVERAGE

3

RTC

Async soft-inpaint for delayed
control loops (2506.07339) address demo manifold narrowness

HIGH
LEVERAGE

4

Recovery / VI

Intervention + verbal coaching
address demo manifold narrowness

HIGH
LEVERAGE

5

Cross-emb + co-train

Web/HL when scene diversity
limited vs real homes

HIGH
LEVERAGE

Apps: new-home cleaning · language tidying · coachable robots · OpenPI FT on DROID/LIBERO/custom

Apps + caution · Visionary north star

Applications

- ▶ New-home mobile cleaning
- ▶ Language-conditioned tidying
- ▶ Coachable robots (VI)
- ▶ OpenPI FT: DROID / LIBERO / custom
- ▶ Richer prompts / longer memory (paper Discussion)

Caution

- ▶ Real-home success $\neq$ solved BEHAVIOR
- ▶ BDDL goals · depth · 100-task multitask
- ▶ Hard handles · occlusion · HL loops
- ▶ Private MM recipe unreproducible
- ▶ Still clearest shared substrate for '26

One-slide message

Open-world household VLA is a co-training + hierarchy problem,
not pure scaling. 2025 operationalized the backbone; 2026 should stress-test
held-out semantics, memory, and teachable HL — GPU-honest, not private-home cosplay.

π_0 VS $\pi_{0.5}$ · what changed

π_0 (2410.24164)

- FM action expert throughout
- No FAST pre-phase
- No unified HL head in recipe
- Lab / matched scenes focus
- ~10k h cross-embodiment
- H=50 · ~10 Euler · 20-50 Hz
- PaliGemma + ~300M expert

$\pi_{0.5}$ (2504.16054)

- FAST pretrain → FM post-train
- FM RETAINED at inference
- Unified HL subtask → LL chunk
- Open-world new homes KPI
- ~400h MM / ~100 homes + mix
- H=50 @ 50 Hz · 10 FM steps
- Co-train: MM ME CE HL WD VI

Novelty = mixture curriculum + hierarchy + hybrid recipe + home-scale eval — not new FM math

Fig.3 · Two-stage: FAST pretrain → FM post-train

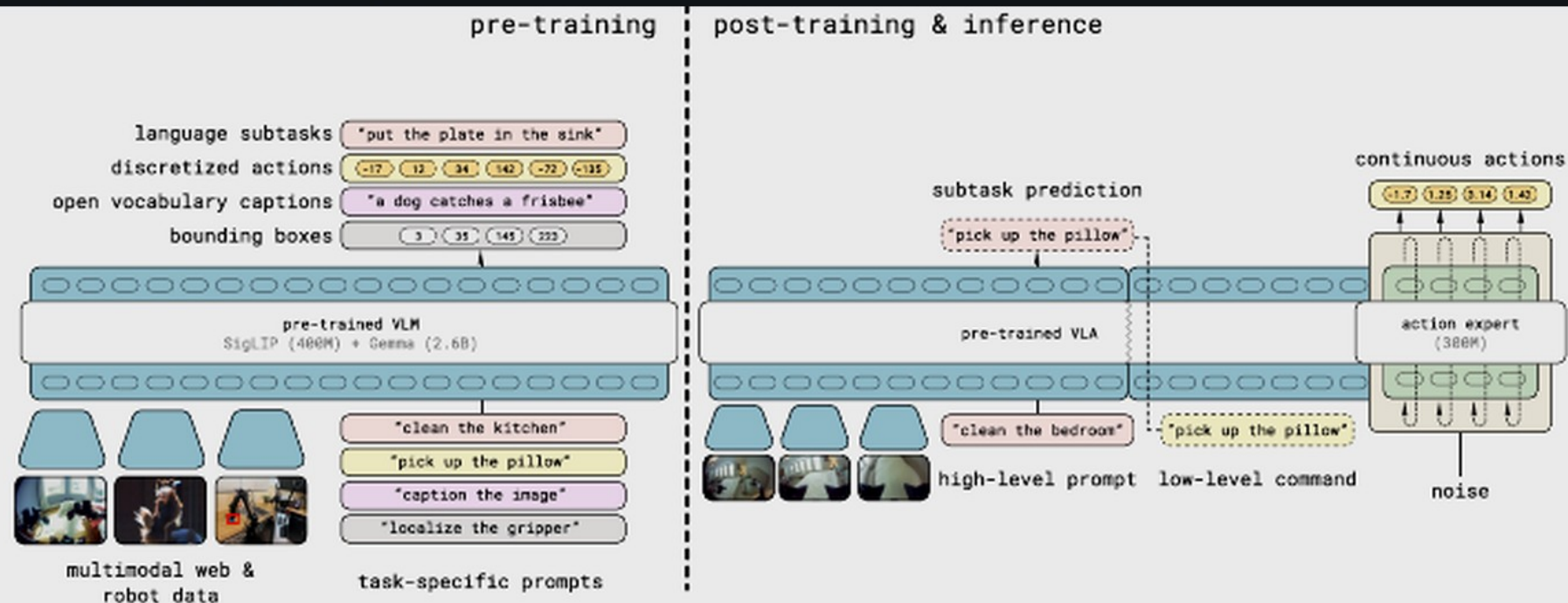


Fig. 3: **Model overview.** $\pi_{0.5}$ is trained in two stages. First, a pre-training stage combines all of the different data sources to produce an initial VLA with discrete tokens. This stage uses data from diverse robotic platforms, high-level semantic action prediction, and data from the web. Robotic data uses the FAST action tokenizer to represent actions as discrete tokens [64]. Second, a post-training stage specializes the model for low-level and high-level inferences for mobile manipulation, leveraging the most task-relevant data, including verbal instructions from human supervisors. This stage uses flow matching to represent the action distribution, enabling efficient real-time inference and the ability to represent fine-grained continuous action sequences. At inference time, the model first infers a high-level subtask, and then predicts the actions based on this subtask.

primitives, such as picking up objects, methods that allow for task-specific assumptions (e.g., grasp prediction, or incorporating model-based planning and control) have been shown to generalize broadly, even to entirely new homes [40, 20, 60, 56, 29]. However, such methods do not readily generalize to the full range of possible tasks that a generalist robot might need to perform. More recently, large-scale datasets collected across

input to output tokens. The weights of these models are initialized from pre-trained vision-language models. By encoding policy inputs and outputs into tokenized representations, the imitation learning problem described above can be cast as a simple next-token-prediction problem over a sequence of observation, instruction and action tokens, and we can leverage the scalable tools of modern machine learning to

$\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization

Physical Intelligence

Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, Ury Zhilinsky

<https://pi.website/blog/pi05>



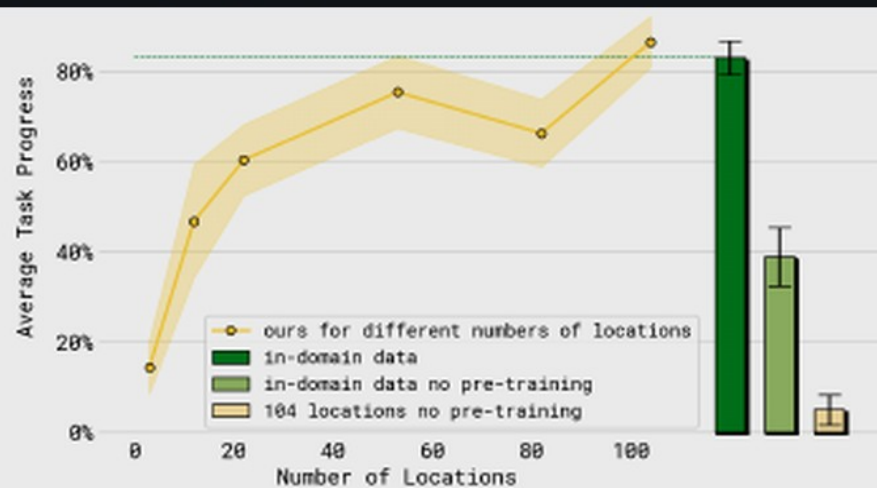


Fig. 8: **Evaluating performance with different numbers of locations.** Performance over the four test tasks — “dishes in sink”, “items in drawer”, “laundry basket”, “make bed” — improves with more training environments. The dashed green line and green bar show a baseline model that includes the test homes in the training set. Compared to this model, our best model achieves similar performance, despite not seeing any data from the test homes.

those in the training data (but new instances), as well as out-of-distribution objects from unseen categories. The latter necessitates broad semantic generalization.

The results of the first experiment are shown in Figure 8. The average performance among the tasks generally improves with more training locations. To quantify how much the final model (with 104 locations) bridges the generalization gap, we include a control (shown in green) that is trained directly on data from the test homes. This control attains similar performance as the final 104-location model, suggesting that our co-training recipe effectively enables broad generalization, reaching similar performance to a model trained on the test

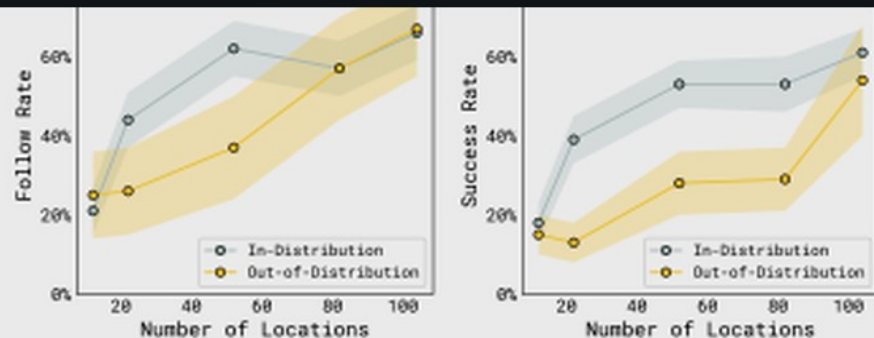


Fig. 9: **Evaluating language following with different numbers of training locations.** We evaluate language following rate and success rate for picking up user-indicated items and placing them into drawers or sinks, averaged over seen object categories (“in-distribution”) or unseen categories (“out-of-distribution”). Performance increases steadily as we increase the number of training locations.

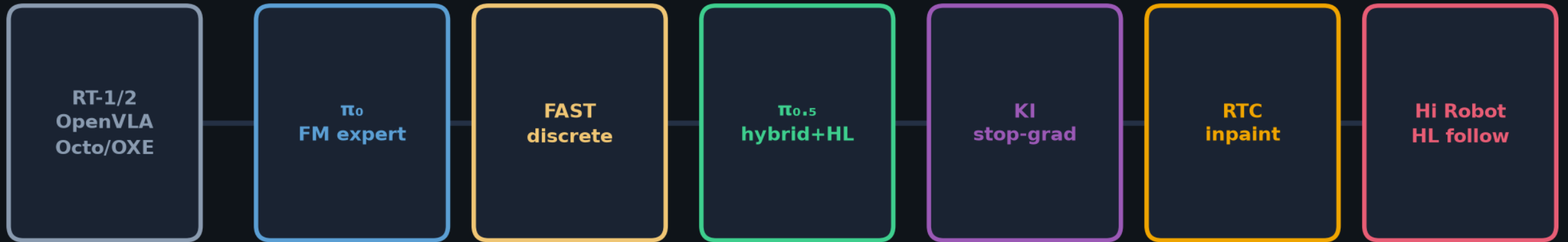
of locations in the training data increases, both language following performance and success rate improve. As expected, the performance on in-distribution objects improves more quickly than that of out-of-distribution objects. As each new environment introduces new household items, the model becomes generally more robust and starts to generalize to task categories that were not present in the training data.

C. How important is each part of our co-training recipe?

To study Question (3), we compare our full $\pi_{0.5}$ model to other training mixtures to study the importance of each mixture component, again using end-to-end task performance in the mock homes and the language following evaluation described in Section V-B. As a reminder, our full recipe uses data from mobile manipulators in many environments (MM), static manipulators in many environments (ME), and diverse

Lineage timeline · Archaeologist spine

RT/OpenVLA/Octo/OXE $\rightarrow$ π_0 $\rightarrow$ FAST $\rightarrow$ $\pi_{0.5}$ $\rightarrow$ KI $\rightarrow$ RTC $\rightarrow$ Hi Robot



Challenge archaeology

BEHAVIOR-1K

household sim foundation

BEHAVIOR'25

50 tasks · $\pi_{0.5}$ winner ~26% q

BEHAVIOR'26

100 tasks · $\pi_{0.5}$ + GR00T baselines

Visionary · close

π 0.5-baseline + targeted deltas — not greenfield.

Keep FM · hierarchy/stages · RTC · recovery/VI · cross-emb+co-train · skip Discord/Drive

Q&A · Sep 30 · π 0.5 / 2504.16054

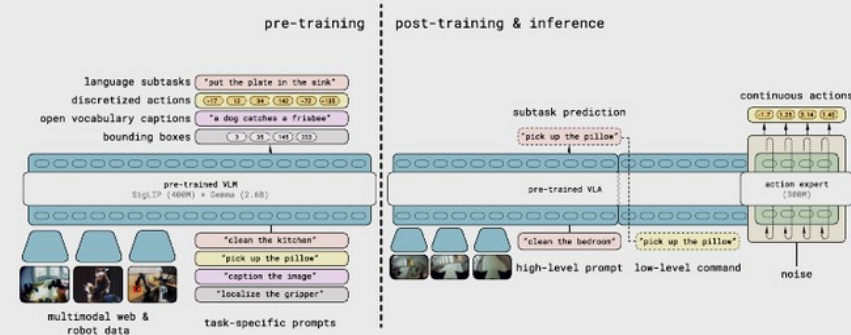


Fig. 3: **Model overview.** $\pi_{0.5}$ is trained in two stages. First, a pre-training stage combines all of the different data sources to produce an initial VLA with discrete tokens. This stage uses data from diverse robotic platforms, high-level semantic action prediction, and data from the web. Robotic data uses the FAST action tokenizer to represent actions as discrete tokens [64]. Second, a post-training stage specializes the model for low-level and high-level inferences for mobile manipulation, leveraging the most task-relevant data, including verbal instructions from human supervisors. This stage uses flow matching to represent the action distribution, enabling efficient real-time inference and the ability to represent fine-grained continuous action sequences. At inference time, the model first infers a high-level subtask, and then predicts the actions based on this subtask.

primitives, such as picking up objects, methods that allow for task-specific assumptions (e.g., grasp prediction, or incorporating model-based planning and control) have been shown to generalize broadly, even to entirely new homes [40, 20, 60, 56, 29]. However, such methods do not readily generalize to the full range of possible tasks that a generalist robot might need to perform. More recently, large-scale datasets collected across many domains [41, 68, 63, 67, 14, 49] have been shown to enable generalization of simple but end-to-end learned tasks to new environments [33, 31, 67, 69, 26, 49, 28, 64]. However, the tasks in these demonstrations are still relatively simple, typically less than a minute in length and often with relatively low success rates. We show that $\pi_{0.5}$ can perform long, multi-stage tasks, such as putting all of the dishes in the sink or picking all of the clothing off the floor of a new bedroom, while generalizing to entirely new homes.

III. PRELIMINARIES

Vision-language-action models (VLAs) are typically trained via imitation learning on diverse robot demonstration datasets $\mathcal{D}$, by maximizing the log-likelihood of an action $\mathbf{a}_t$ (or, more generally, an action *chunk* $\mathbf{a}_{t:t+H}$) given an observation $\mathbf{o}_t$ and a natural language task instruction ℓ : $\max_{\theta} \mathbb{E}_{(\mathbf{a}_{t:t+H}, \mathbf{o}_t, \ell) \sim \mathcal{D}} \log(\pi_{\theta}(\mathbf{a}_{t:t+H} | \mathbf{o}_t, \ell))$. The observation typically contains one or more images $\mathbf{I}_1^t, \dots, \mathbf{I}_n^t$ and proprioceptive state $\mathbf{q}_t$, which captures the position of the robot's joints. VLA architectures follow the design of modern language and vision-language models, with modality-specific tokenizers that map inputs and outputs to discrete ("hard") or continuous ("soft") token representations, and a large, autoregressive transformer backbone that is trained to map from

input to output tokens. The weights of these models are initialized from pre-trained vision-language models. By encoding policy inputs and outputs into tokenized representations, the imitation learning problem described above can be cast as a simple next-token-prediction problem over a sequence of observation, instruction and action tokens, and we can leverage the scalable tools of modern machine learning to optimize it. In practice, the choice of tokenizers for image and text inputs follows those of modern vision-language models. For actions, prior work has developed effective, compression-based tokenization approaches [64], which we use in this work during pretraining. A number of recent VLA models have also proposed to represent the action distribution via diffusion [55, 84, 52] or flow matching [8], providing a more expressive representation over continuous-valued action chunks. During the post-training phase of our model, we will build on the design of the π_0 model [8], which represents the action distribution via flow matching. In this design, the tokens corresponding to actions receive the partially denoised actions from the previous step of flow matching as input, and output the flow matching vector field. These tokens also use a different set of model weights, which we refer to as an "action expert," analogously to a mixture of experts architecture. This action expert can specialize to flow matching-based action generation, and can be significantly smaller than the rest of the LLM backbone.

IV. THE $\pi_{0.5}$ MODEL AND TRAINING RECIPE

We provide an overview of the $\pi_{0.5}$ model and training recipe in Figure 3. The model weights are initialized from a standard VLM trained on data from the web, and training then